

Relatório de Análise de Treliça

Resultados Obtidos:

Forças nas Barras: [-288.68, -288.68, 577.35, 577.35, -1000.0]
Reações de Apoio: [0.0, -500.0, -500.0]

Código de Cálculo

```
import numpy as np
import matplotlib.pyplot as plt

def resolver_treliga():
    # 1. ENTRADAS
    nos_coords = np.array([
        [0.0, 0.0], # Nó 0
        [2.0, 0.0], # Nó 1
        [1.0, 1.732], # Nó 2
        [1.0, 0.0] # Nó 3
    ])
    N = len(nos_coords)

    barras = np.array([
        [0, 3], [3, 1], [0, 2], [2, 1], [2, 3]
    ])
    B = len(barras)

    F = np.zeros(2 * N)
    F[2*3 + 1] = -1000 # -1000N no Nó 3 (direção Y) A

    K = np.zeros((2*N, 2*N))

    for i, (n1, n2) in enumerate(barras):
        dx = nos_coords[n2][0] - nos_coords[n1][0]
        dy = nos_coords[n2][1] - nos_coords[n1][1]
        L = np.sqrt(dx**2 + dy**2)
        cos, sin = dx/L, dy/L

        A[2*n1, i] = cos
        A[2*n1+1, i] = sin
        A[2*n2, i] = -cos
        A[2*n2+1, i] = -sin

    # Reações (Apoio fixo no Nó 0, móvel no Nó 1)
    A[2*0, B] = 1 # R_ax
    A[2*0+1, B+1] = 1 # R_ay
    A[2*1+1, B+2] = 1 # R_by
```

```

def gauss_elimination(A_orig, b_orig):
    n = len(b_orig)
    M = np.hstack([A_orig, b_orig.reshape(-1, 1)])
    for i in range(n):
        max_el = abs(M[i, i])
        max_row = i
        for k in range(i + 1, n):
            if abs(M[k, i]) > max_el:
                max_el = abs(M[k, i])
                max_row = k
        M[i], M[max_row] = M[max_row].copy(), M[i].copy()
        for k in range(i + 1, n):
            c = -M[k, i] / M[i, i]
            M[k, i:] += c * M[i, i:]
    res = np.zeros(n)
    for i in range(n - 1, -1, -1):
        res[i] = (M[i, n] - np.dot(M[i, i+1:n], res[i+1:n])) / M[i, i]
    return res

solucao = gauss_elimination(A, F)
# ... (processamento e plotagem)

```

Visualização Gráfica

